

API Alerts - AlertController

Projet: Urban Alert - Spring Boot API
Base URL: `http://localhost:8080/api/v1/alerts`
Context path: `/api` (voir `application.properties`)

Authentification et sécurité

Règle | Détail

Endpoints `/v1/alerts/**` | **JWT obligatoire** (`Authorization: Bearer <token>`) via `anyRequest().authenticated()`

Sans token | `401 Unauthorized` (filtre Spring Security)

Rôle pour **créer** une alerte | **Aucune restriction de rôle** dans le code métier : tout utilisateur authentifié peut créer une alerte (CITOYEN, AGENT, SUPER_AGENT, ADMIN)

Cas d'usage principal | **CITOYEN** signale un incident ; l'alerte est rattachée à l'utilisateur connecté

Changement de statut manuel | **Interdit** pour `CITOYEN` ; autorisé pour `AGENT`, `SUPER_AGENT`, `ADMIN`

Note: `SecurityConfig` autorise sans auth `/api/alerts/**` (ancien chemin). Le contrôleur actuel est sous `/v1/alerts`, donc protégé par JWT.

Qui peut créer une alerte ?

Rôle | Créer (`POST /v1/alerts`) | Modifier / supprimer la sienne | Changer statut (`PATCH ?/status`)

CITOYEN | Oui (authentifié) | Oui, si statut `NEW` et propriétaire | **Non**

AGENT | Oui | Oui, si statut `NEW` et propriétaire | Oui

SUPER_AGENT | Oui | Oui, si statut `NEW` et propriétaire | Oui

ADMIN | Oui | Oui, si statut `NEW` et propriétaire | Oui

Non authentifié | **Non** | **Non** | **Non**

Règles métier création:

- L'alerte est créée avec `status = NEW` (mapper / `@PrePersist`).
- `user_id` = utilisateur connecté (pas choisi dans le body).
- La catégorie (`categoryId`) doit exister (`problem_types`).

Modification / suppression:

- Uniquement le **créateur** de l'alerte (`findByIdAndUserId`).
- Uniquement si statut **NEW** (sinon `InvalidAlertException` -> HTTP 409).

Médias (images/vidéos):

- Pas de vérification du propriétaire dans le service : tout utilisateur **authentifié** peut ajouter/retirer des médias sur **n'importe quelle** alerte si l'ID existe.

Statuts et priorités

AlertStatusEnum: `NEW`, `IN_PROGRESS`, `RESOLVED`, `REJECTED`

AlertPriorityEnum: `LOW`, `MEDIUM`, `HIGH`

Le statut peut aussi passer à `IN_PROGRESS` / `RESOLVED` / `REJECTED` via un **Problem** (agrégation), sans appeler `PATCH /status`.

Format des erreurs (controller)

```
{
  "error": "Titre court",
  "message": "Détail",
  "timestamp": 1717000000000
}
```

Erreurs de validation `@Valid` (champs manquants) : gérées par Spring (souvent **400** avec détail des champs, hors `createErrorResponse` du controller).

DTO - champs requis / optionnels

CreateAlertRequestDTO - `POST /v1/alerts`

Champ | Requis | Contraintes
`title` | **Oui** | 3?255 caractères, non vide
`description` | **Oui** | 10?5000 caractères, non vide
`latitude` | **Oui** | -90 à 90
`longitude` | **Oui** | -180 à 180
`categoryId` | **Oui** | ID `problem\_types` existant
`priority` | **Oui** | LOW, MEDIUM ou HIGH
`address` | Non | max 500 caractères
`isAnonymous` | Non | défaut `false`

Exemple:

```
{
  "title": "Nid de poule avenue Hassan II",
  "description": "Très dangereux pour les voitures près du rond-point.",
  "latitude": 33.5731,
  "longitude": -7.5898,
  "address": "Casablanca, Maarif",
  "categoryId": 1,
  "priority": "HIGH",
  "isAnonymous": false
}
```

Réponse: `AlertResponseDTO` - statut initial `NEW`, pas d'images/vidéos à la création.

UpdateAlertRequestDTO - `PUT /v1/alerts/{id}`

Tous les champs sont **optionnels** (mise à jour partielle). Si présents, mêmes contraintes de taille que à la création.

Champ | Requis
`title` | Non
`description` | Non
`latitude` | Non
`longitude` | Non
`address` | Non
`categoryId` | Non
`priority` | Non
`isAnonymous` | Non

Conditions: créateur + statut `NEW`.

AddMediaRequestDTO - `POST /{id}/images` et `POST /{id}/videos`

Champ | Requis | Contraintes
`mediaUrl` | **Oui** | non vide, max 2048
`description` | Non | -

DELETE médias - query params

Paramètre | Requis
`imageUrl` | **Oui** (`DELETE /{id}/images`)
`videoUrl` | **Oui** (`DELETE /{id}/videos`)

AlertResponseDTO (réponse)

Champ | Description
id, title, description, latitude, longitude, address | Données alerte
status, priority, isAnonymous | Enums / booléen
images, videos | Listes d'URLs (JSON en base)
createdAt, updatedAt | Dates
user | UserSummaryResponseDTO (créateur)
category | ProblemTypeResponseDTO
ticketId | Optionnel (legacy)

****Non exposé actuellement:**** `problemId` (relation Problem).

Endpoints détaillés

1. CRUD

POST `/v1/alerts` - Créer

- ****Auth:**** JWT, tout rôle.
- ****Body:**** `CreateAlertRequestDTO` (voir tableau ci-dessus).
- ****Succès:**** `201`

HTTP | Exception / cause

404 | `UserNotFoundException` (utilisateur JWT introuvable)

400 | `ProblemTypeNotFoundException` (catégorie), autres erreurs, validation

401 | Non authentifié

GET `/v1/alerts/{id}` - Détail

- ****Path:**** `id` requis.
- ****Body:**** aucun.
- ****Succès:**** `200`

HTTP | Exception

404 | `AlertNotFoundException`

GET `/v1/alerts` - Liste (pagination)

- ****Query:**** `page` (défaut 0), `size` (défaut 10).
- ****Auth:**** JWT.
- ****Succès:**** `200` (page d'alertes).

HTTP | Exception

500 | Erreur générique

GET `/v1/alerts/user/my-alerts` - Mes alertes

- ****Query:**** `page`, `size`.
- ****Auth:**** JWT - alertes du user connecté.
- ****Succès:**** `200`

HTTP | Exception

404 | `UserNotFoundException`

PUT `/v1/alerts/{id}` - Modifier

- ****Path:**** `id`.
- ****Body:**** `UpdateAlertRequestDTO` (tous optionnels).
- ****Auth:**** créateur uniquement + statut `NEW`.
- ****Succès:**** `200`

HTTP | Exception

404 | `AlertNotFoundException` (id inconnu ou pas le propriétaire)

409 | `InvalidAlertException` (statut ? NEW)

400 | Autres erreurs ; `ProblemTypeNotFoundException` si `categoryId` invalide (non capturée -> 400 générique)

DELETE `/v1/alerts/{id}` - Supprimer

- ****Path:**** `id`.
- ****Body:**** aucun.
- ****Auth:**** créateur + statut `NEW`.
- ****Succès:**** `204`

HTTP | Exception
404 | `AlertNotFoundException`
409 | `InvalidAlertException` (statut ? NEW)

2. Médias

POST `/v1/alerts/{id}/images` | `/videos`

- **Body:** `AddMediaRequestDTO` (`mediaUrl` requis).
- **Succès:** `201`

HTTP | Exception
404 | `AlertNotFoundException`
400 | `InvalidAlertException`, validation, erreur JSON

DELETE `/v1/alerts/{id}/images?imageUrl=?` | `/videos?videoUrl=?`

- **Query:** URL requise.
- **Succès:** `200`

HTTP | Exception
404 | `AlertNotFoundException`
400 | `InvalidAlertException` (URL absente de la liste)

3. Recherche et filtres

Méthode | URL | Paramètres
GET | `/category/{categoryId}` | path: categoryId ; query: page, size
GET | `/status/{status}` | path: NEW, IN_PROGRESS, RESOLVED, REJECTED ; query: page, size
GET | `/search` | query: **keyword** (requis), page, size

Succès: `200` - **500** en cas d'erreur serveur.

4. Gestion du statut

PATCH `/v1/alerts/{id}/status?status=?`

- **Query:** `status` (**requis**) - enum.
- **Auth:** AGENT, SUPER_AGENT ou ADMIN (pas CITOYEN).
- **Succès:** `200`

HTTP | Exception
403 | `SecurityException` (CITOYEN)
404 | `UserNotFoundException`, `AlertNotFoundException`
400 | Autres erreurs

GET `/v1/alerts/{id}/status`

- **Succès:** `200` - `{ "alertId": 1, "status": "NEW" }`
- **404:** `AlertNotFoundException`

5. Statistiques

Méthode | URL | Paramètres | Réponse
GET | `/stats/count-by-status` | `status` (requis) | `{ status, count }`
GET | `/stats/user-count` | - (user connecté) | `{ userId, count }`

Exceptions: `404` UserNotFoundException sur user-count ; `500` sur count-by-status.

Liste des exceptions (package `exception.alert` et liées)

Exception | Quand | HTTP typique (controller)

`AlertNotFoundException` | ID alerte introuvable ou pas propriétaire (update/delete) | 404
`InvalidAlertException` | Statut ? NEW (update/delete) ; média introuvable ; erreur ajout média | 409 ou 400
`UserNotFoundException` | User JWT / id introuvable | 404
`ProblemTypeNotFoundException` | `categoryId` invalide à la création | 400 (catch générique)
`SecurityException` | CITOYEN tente PATCH status | 403

Récapitulatif permissions

CREER ALERTE -> Tout utilisateur authentifié (surtout CITOYEN)
MODIFIER/SUPPRIMER -> Createur + statut NEW uniquement
PATCH STATUS -> AGENT | SUPER_AGENT | ADMIN
LECTURE liste -> Authentifié
MEDIAS -> Authentifié (pas de controle propriétaire actuellement)

Document genere a partir de AlertController.java, DTOs alert/ et AlertServiceImpl.java.*